Sistemas Operativos I

Introducción

Objetivo de la clase de hoy: revisitar la historia y presentar la materia.

Qué es un Sistema Operativo?

Un sistema operativo es uno o varios programas que se usan para poder trabajar con los componentes de un equipo de cómputo.

Los sistemas operativos proveen un conjunto de funciones necesarias usadas por diversos programas de aplicaciones de una computadora, y los vínculos necesarios para controlar y sincronizar el hardware de la misma.

En las primeras computadoras, que no tenían sistema operativo, cada programa necesitaba la más detallada especificación del hardware para ejecutarse correctamente y desarrollar tareas estándares, y sus propios drivers para los dispositivos periféricos como impresoras y lectores de tarjetas perforadas. El incremento de la complejidad del hardware y los programas de aplicaciones eventualmente hicieron del sistema operativo una necesidad.

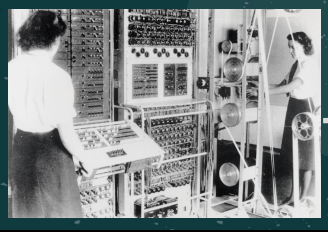
Funciones y objetivos del SO

- **Abstracción:** Los programas no deben tener que preocuparse de los detalles de acceso a hardware, o de la configuración particular de una computadora.
- **Administración de recursos:** Los recursos son la memoria, espacio de almacenamiento, tiempo de procesamiento, etc. Los procesos compiten por estos recursos. El SO tiene que gestionarlo de manera efectiva.
- **Aislamiento:** En un sistema multiusuario y multitarea cada proceso y cada usuario no tendrá que preocuparse por otros que estén usando el mismo sistema.

En las primeras computadoras, que no tenían sistema operativo, cada programa necesitaba la más detallada especificación del hardware para ejecutarse correctamente y desarrollar tareas estándares, y sus propios drivers para los dispositivos periféricos como impresoras y lectores de tarjetas perforadas. El incremento de la complejidad del hardware y los programas de aplicaciones eventualmente hicieron del sistema operativo una necesidad.

Historia Sistemas Operativos

- 1962 : Canales
- A finales de los 60': Procesadores con múltiples unidades de procesamiento.
- 1960-1970: Región Crítica + Sincronización
- Finales 1970 y Ppio de 1980: Comunicaciones y redes
- 1980 - 1990 : Computación Distribuida
- Finales del siglo pasado: grandes cantidades de unidades de procesamiento, modelo cliente/servidor, Internet y WWW!!!!



Veamos **una** historia de cómo presentar la programación concurrente y paralela.

La programación concurrente nace en 1962 con la invención de los denominados *canales*. Controladores independientes que permitían al CPU ejecutar un programa mientras que al mismo tiempo se podían realizar operaciones de entrada/salida (IO) sobre un procesos suspendido que estaba a la espera de las mismas. Es decir, se podía tener la idea de una ejecución al mismo tiempo de dos procesos. Nace de la interacción del CPU con el mundo exterior, donde pasamos de tener procesos totalmente cerrados a poder interactuar con el CPU. Ya que el Sistema operativo es el programa encargado de establecer una interfaz entre el hardware y el usuario, la programación concurrente se presenta como una problemática de los sistemas operativos. Aunque como ya verán en otras materias (EDyAII) hoy en día ya es un campo en sí mismo.

A finales de 1960 ya se producen los primeros procesadores con múltiples unidades de procesamiento cumpliendo por fin la promesa de tener varios procesos ejecutándose al mismo tiempo, dando lugar a lo que se conoce como computación paralela.
El primer problema que se encuentra entonces es lo que se conoce como *el problema de la región crítica*. Ahora se tienen varios procesos compartiendo su tiempo de vida, y nace la pregunta de cómo compartir los recursos disponibles entre ellos? Por ejemplo, si hay una impresora, ¿quién y cómo se tiene acceso a ella? Todos los procesos tienen el mismo nivel de precedencia?
Por ejemplo: imaginemos una computadora con una sóla unidad de procesamiento,

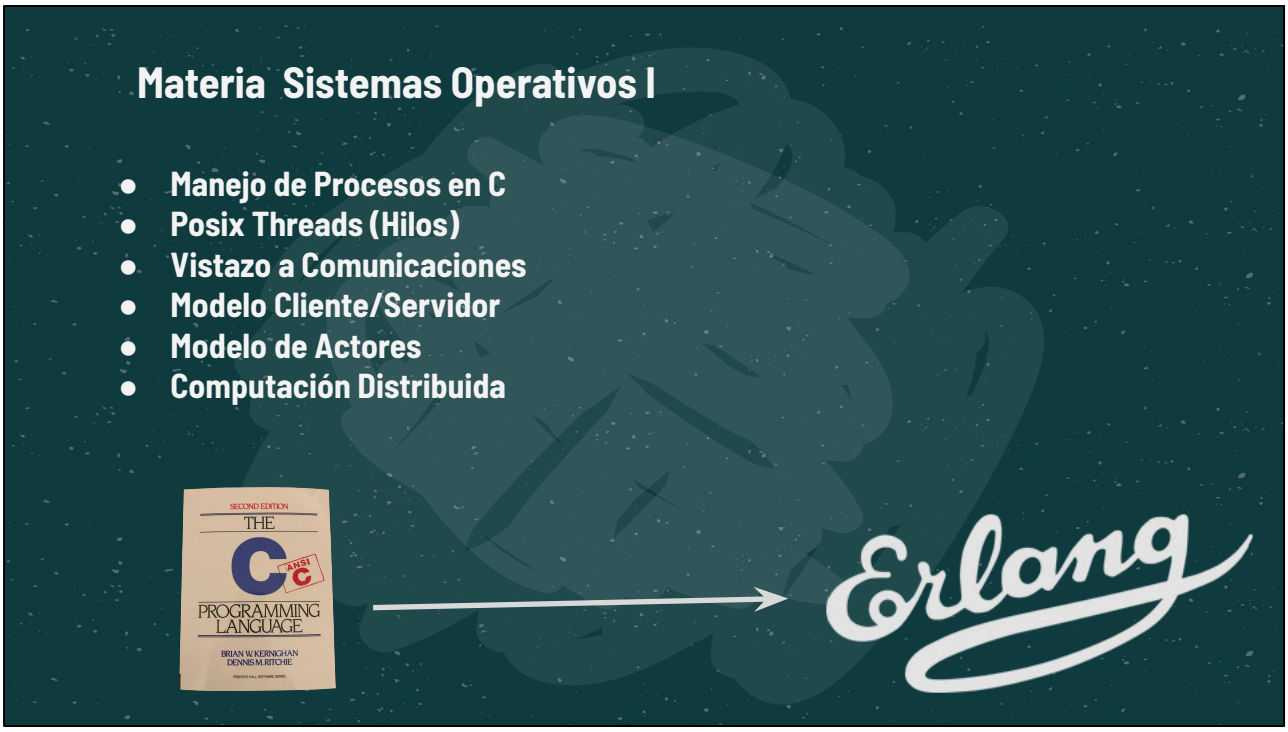
por lo que todos los procesos comparten dicha unidad. Ahora es posible que queramos que los procesos dedicados al sistema operativo tengan mayor privilegio. Se introduce entonces el concepto de sincronización: ¿podemos organizar los procesos de manera tal que todos accedan a los recursos de una manera *eficiente?*. Y para esto se diseñaron distintos tipos de mecanismos como ser: semáforos, barreras, monitores, etc.

A finales de 1970 y principio de 1980 el foco se mueve al mundo de las comunicaciones y las redes, con proyectos como **Arpanet** y **Ethernet.** Tener varias computadoras conectadas en red da lugar a una nueva forma de computación denominada *computación distribuida*, donde obliga a repensar el modelo de computación como diferentes actores enviando mensajes en vez de leer y escribir en segmentos de memoria compartida (como lo era hasta ese momento).

Entre 1980 y 1990 el foco de estudio se centró en el estudio de sistemas distribuidos. Llegando entonces a finales del siglo pasado pasado donde se tienen ya grandes cantidades de unidades de procesamiento dedicadas a resolver problemas concretos, **INTERNET, WWW** y el modelo cliente/servidor. Como además el ingreso de las computadoras a la vida cotidiana de las personas.

Imagen Izquierda: Colossus computer (1943-1945)
https://en.wikipedia.org/wiki/Colossus_computer
Colossus fue la primera computadora digital electrónica del mundo que era programable. Colossus utilizó válvulas termoiónicas (tubos de vacío) para realizar operaciones y cálculos booleanos.

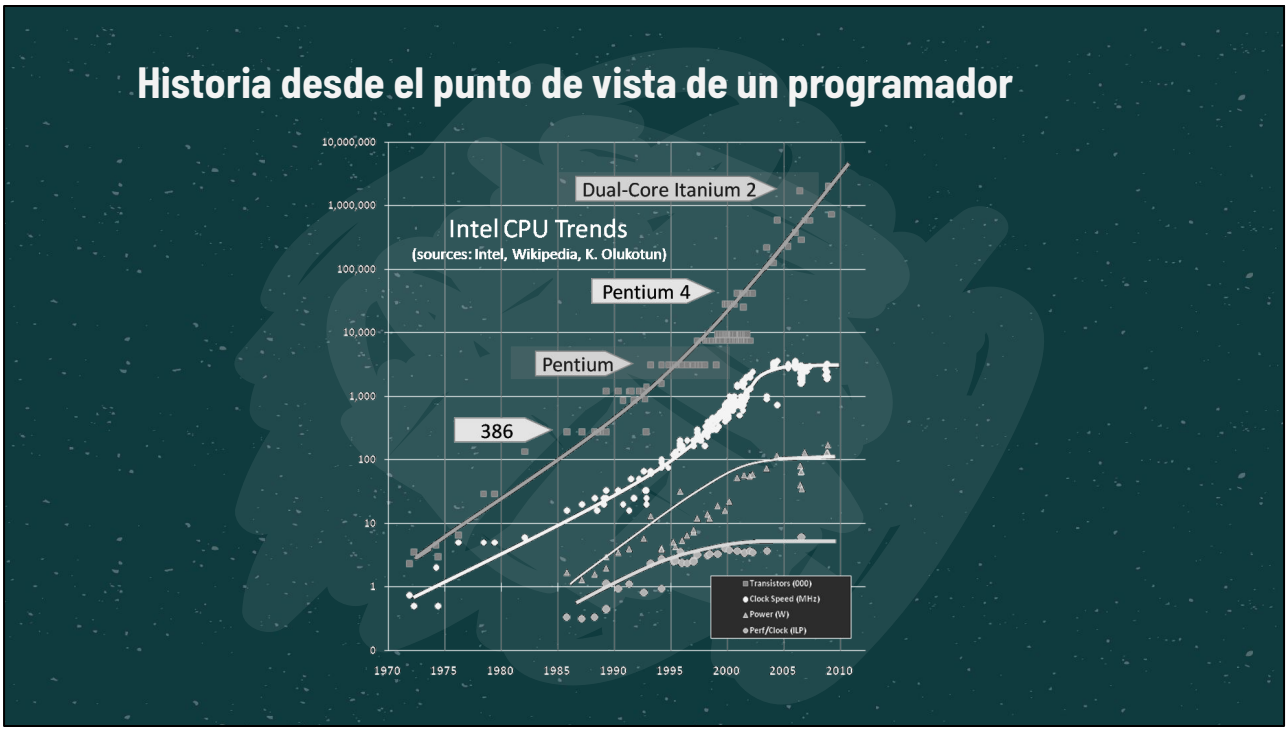
Imagen derecha: Computadora BTM (ICT) HEC 1202 (1953-1956)
http://www.chilton-computing.org.uk/acl/literature/reports/p017.htm



Desde la materia seguiremos el mismo recorrido que la historia nos marcó.
Veremos (al menos) dos lenguajes de programación, partiendo la materia a la mitad.
Primero utilizaremos el lenguaje de programación C donde partiremos desde lo más cerca posible del Sistema Operativo: llamada a sistema y creación de procesos. Y a su vez comenzaremos a estudiar los problemas de región crítica (como memoria compartida) y sincronización.
Luego, pasaremos a estudiar el concepto de Hilos utilizando el estándar de Posix Threads como alternativa a la creación de procesos.
Donde concluimos la etapa de C estudiando comunicación entre procesos, visitando un poco el área de comunicaciones y presentaremos el modelo cliente-servidor.

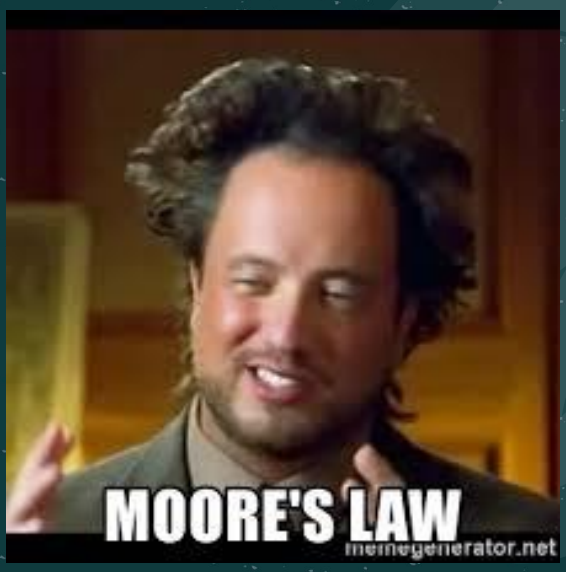
Luego pasaremos a estudiar otro lenguaje, Erlang, que se basa en la idea que la computación se produce mediante la comunicación entre los diferentes procesos. Esto nos lleva a repensar la forma en la que estructuramos los programas, y nos vamos a divertir con eso.

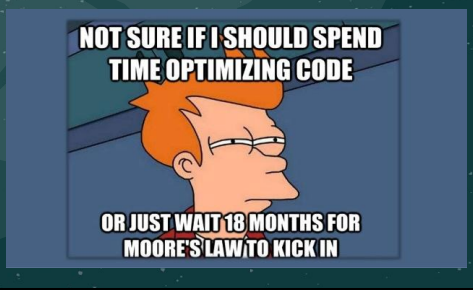
En otras palabras, partiremos desde los conceptos más crudos e iremos creciendo en abstracción.



La realidad desde el punto de vista del programador la historia fue distinta.
Recién los procesadores de múltiples cores llegaron a principios de los 2000.

Historia desde el punto de vista de un programador



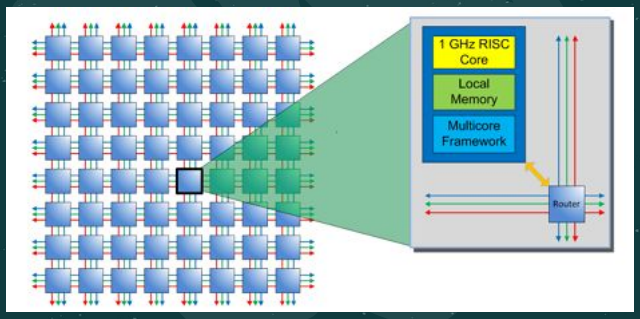


Durante todo ese periodo estaba instaurada lo que se conoce como Ley de Moore que dice a grandes rasgos que el procesamiento y memoria ram se duplica a razón de dos años.
Moore se basó en una técnica presentada por Dennard en 1974, que describe básicamente como duplicar la cantidad de transistores de un procesador a costa de aumentar un poco el consumo energético.
Entonces para qué voy a adaptar mi código usando programación concurrente/paralela/distribuida si simplemente mi código mágicamente va a andar más rápido, **exactamente el mismo código**, cuando salgan la nueva generación de CPUs.

Bueno lamentablemente esa historia terminó trágicamente, no es gratis duplicar la cantidad de transistores en (casi) el mismo espacio físico. Y aquí es donde viene la física a darnos una lección: ningún proceso es perfecto.
Siempre hay algo que energía que se pierde, y en éste caso, en forma de **calor**.
A finales del 2005, Intel ya abandona la producción de procesadores de un sólo núcleo.

"No such thing as a free lunch" (also written as "There ain't no such thing as a free lunch" and sometimes called Crane's law) is a popular adage communicating the idea that it is impossible to get something for nothing.

Solución: Procesadores con varios núcleos



La solución del hardware es entonces pasar a tener múltiples unidades de procesamiento en el mismo procesador, permitiendo ejecutar múltiples procesos en exactamente el mismo tiempo. **Obligando** entonces a los programadores y la industria del software a repensar el software de forma Paralela. Esto es porque dos núcleos de 3Ghz no es lo mismo que uno sólo de 6Ghz.
La programación paralela comparte algunas problemáticas de la programación concurrente pero no es exactamente lo mismo.
Dentro de la materia no vamos a explorar extensamente el desarrollo de programas paralelos, eso lo verán con más detalle en la materia de EDyA2.